

01

02

03

04

05

06

07

08

09

10

11

12

13

14

15

16

17

18

19

20

Aula 6

Observabilidade em APIs

Health checks, logs, métricas e traces no contexto de APIs e microsserviços.

latencia

ultimos 15 min

status

UP

health ok

erro 5xx

0.4%

monitorar

Objetivo da aula

Parar de adivinhar e começar a investigar com dados.

A aplicacao precisa responder: estou saudavel, estou lenta, estou errando e onde o problema comecou?

Quando o sistema cresce

Cliente

chama a API

Gateway

encaminha

Pedidos

processa regra

MongoDB

salva dados

RabbitMQ

entrega evento

monitoramento / sintomas conhecidos

● alertas

sinais, a equipe tenta descobrir no escuro.

Monitoramento

1 / 20

Proximo

Acompanha sinais que ja sabemos que importam.

CPU

62%

normal

memoria

1.8G

estavel

latencia p95

820ms

alto

alerta

Observabilidade

Ajuda a descobrir por que o sintoma apareceu.

- **Monitoramento:** a latencia subiu.
- **Observabilidade:** o tempo foi gasto na consulta ao MongoDB.
- **Acao:** investigar indice, volume de dados ou consulta lenta.

Logs, metricas e traces

Logs

Eventos importantes com contexto: pedido criado, erro ao buscar, timeout, falha de validacao.

Metricas

Numeros ao longo do tempo: requisicoes, latencia, memoria, CPU, status HTTP.

Traces

Caminho da requisicao entre servicos, banco, filas e chamadas externas.

Log bom conta o que aconteceu

```
10:15:21 INFO      Criando pedido para o cliente ana@email.com
10:15:21 INFO      Pedido criado com sucesso. orderId=662f...
10:16:04 WARN      Pedido nao encontrado. orderId=id-inexistente
10:17:32 ERROR      Timeout ao consultar dependencia externa
```

O log precisa ajudar a investigar. Ele nao deve expor senha, token, documento ou informacao sensivel.

Metricas mostram padroes

Requisicoes por minuto



O que observar

- Quantidade de chamadas.
- Tempo de resposta.
- Status HTTP.
- Memória, CPU e processo.

Trace mostra onde o tempo foi gasto

POST /api/v1/orders	920ms
gateway-service	24ms
orders-service	318ms
validation	6ms
MongoDB save	287ms
RabbitMQ publish	41ms
response mapping	9ms

Nesta aula vamos focar em logs, health e metricas. Tracing fica como evolucao natural quando houver varios servicos conversando.

Endpoints operacionais

A aplicação expõe informações técnicas separadas das regras de negócio.

/health

Mostra se a aplicação e suas dependências estão saudáveis.

/metrics

Permite inspecionar medidas internas da aplicação.

/metrics/prometheus

Entrega métricas em um formato que o Prometheus consegue coletar.

Health check nao e enfeite

application

UP

respondendo

mongodb

UP

conectado

Uma aplicacao pode estar com o processo em execucao e mesmo assim nao conseguir trabalhar. O health check ajuda a enxergar isso.

Instrumentação produz métricas

HTTP

Mede volume, duração, rotas e status das requisições.

Processo

Mede memória, CPU, threads e uso de recursos.

Exportador

Converte as métricas para o formato esperado pela ferramenta de coleta.

Prometheus coleta, nao recebe

```
scrape_configs:  
  - job_name: observability-api  
    metrics_path: /metrics/prometheus  
    static_configs:  
      - targets:  
        - host.docker.internal:8080
```

A cada intervalo, o Prometheus chama a API e guarda as metricas.
Depois conseguimos consultar historico e montar dashboards.

[Anterior](#)

13 / 20

[Proximo](#)

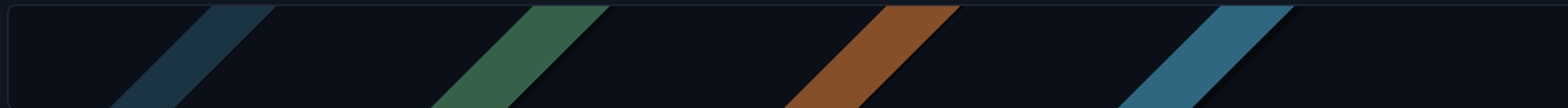
Dashboard home

Dashboard bom responde pergunta

req/s

128

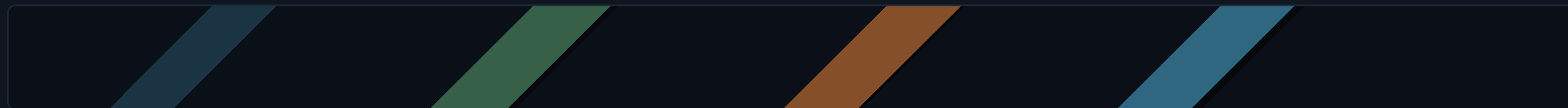
normal



p95

740ms

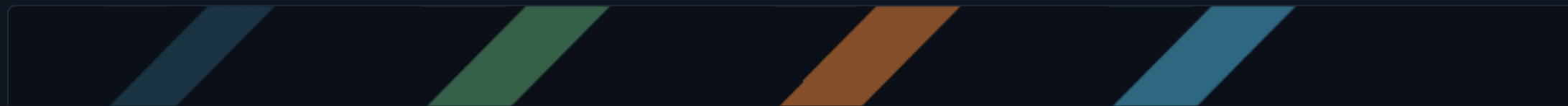
subindo



5xx

1.2%

investigar



Nao observe so CPU e memoria

A aplicacao existe para executar uma regra de negocio.

Pedidos criados

Mostra se o fluxo principal esta acontecendo.

Pedidos com erro

Mostra impacto real para o usuario.

Eventos publicados

Mostra se a integracao continua ativa.

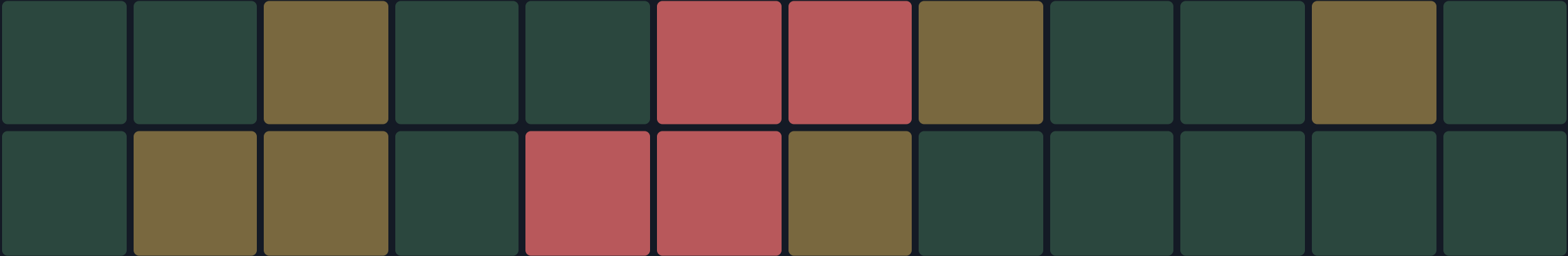
Alerta precisa pedir acao

Ruim

Dispara toda hora, ninguem olha, nao indica o que fazer.

Bom

Mostra impacto, gravidade e aponta o primeiro lugar para investigar.



Erros comuns

- Achar que log sozinho resolve observabilidade.
- Expor todos os endpoints operacionais sem autenticação ou necessidade.
- Criar alertas para tudo e cansar a equipe.
- Registrar dados sensíveis em log.
- Esperar metricas sem gerar trafego na API.

Aplicando os conceitos

API de pedidos

As operações geram tráfego, resultados, erros e eventos que podem ser observados.

Observabilidade básica

Health check, logs estruturados, métricas exportadas e coleta periódica pelo Prometheus.

```
POST /api/v1/orders
GET  /health
GET  /metrics
GET  /metrics/prometheus
```

Depois da base

Quando health, logs e metricas fizerem sentido, tracing passa a ser o proximo passo.

- Correlacionar requisicoes entre microservicos.
- Usar OpenTelemetry para padronizar coleta.
- Montar dashboards com historico e comparacao.
- Definir alertas baseados em impacto real.

Resumo

Monitoramento

Mostra que algo saiu do esperado.

Observabilidade

Ajuda a descobrir por que aquilo aconteceu.

Base operacional

Health checks, instrumentação e coleta de métricas formam um bom começo.

A meta não é ter painel bonito. A meta é conseguir investigar melhor quando o sistema se comporta de forma estranha.

